

Manual de Funciones Reutilizables

Snippets de JavaScript extraídos del proyecto Ruleta. Copia y pega en cualquier página.

Contenido

1. Web Audio API — Sonidos sintetizados
2. Canvas 2D — Dibujar y animar
3. Grabación de video (canvas a MP4)
4. Grabación con audio del juego
5. Sistema de idiomas (i18n)
6. Temas con CSS variables
7. Drag & Drop de archivos
8. Procesamiento de imágenes
9. Persistencia en localStorage
10. Aleatoriedad segura (crypto)
11. Firebase Auth (Google Login)
12. Firebase Firestore (base de datos)



Web Audio API

Sin archivos

Genera sonidos completamente con código JavaScript, sin necesidad de archivos MP3/WAV.

Crear contexto de audio

Requisito: Se debe llamar por primera vez dentro de un gesto del usuario (clic, tecla, toque).

```
let audioCtx = null;
let masterGain = null;

function getAudio() {
  if (!audioCtx) {
    const AC = window.AudioContext || window.webkitAudioContext;
```

Copiar

```
    if (!AC) return null;
    audioCtx = new AC();
    masterGain = audioCtx.createGain();
    masterGain.connect(audioCtx.destination);
  }
  if (audioCtx.state === 'suspended') audioCtx.resume();
  return audioCtx;
}
```

Tick / clic

Sonido corto tipo "tic" para conteos, cronómetros, selecciones.

intensity: 0.0 (suave/agudo) a 1.0 (fuerte/grave).

```
function playTick(intensity = 0.5) {
  const ac = getAudio();
  if (!ac) return;
  const o = ac.createOscillator();
  const g = ac.createGain();
  o.type = 'square';
  o.frequency.value = 500 + 1000 * intensity;
  const t0 = ac.currentTime;
  g.gain.setValueAtTime(0.04 + 0.08 * intensity, t0);
  g.gain.exponentialRampToValueAtTime(0.0001, t0 + 0.05);
  o.connect(g);
  g.connect(masterGain);
  o.start(t0);
  o.stop(t0 + 0.06);
}
```

[Copiar](#)

Whoosh (viento)

Ruido filtrado que simula un "whoosh" de velocidad. Ideal para animaciones de entrada/salida.

duration: duración en segundos (ej: 3.5).

```
function playWhoosh(duration = 3) {
  const ac = getAudio();
  if (!ac) return;
  const len = Math.max(1, Math.floor(ac.sampleRate * duration));
  const buffer = ac.createBuffer(1, len, ac.sampleRate);
  const data = buffer.getChannelData(0);
  for (let i = 0; i < len; i++)
```

[Copiar](#)

```

    data[i] = (Math.random() * 2 - 1) * (1 - i / len);
    const src = ac.createBufferSource();
    src.buffer = buffer;
    const filter = ac.createBiquadFilter();
    filter.type = 'bandpass';
    filter.Q.value = 0.9;
    const t0 = ac.currentTime;
    filter.frequency.setValueAtTime(250, t0);
    filter.frequency.exponentialRampToValueAtTime(3500, t0 + duration * 0.45);
    filter.frequency.exponentialRampToValueAtTime(180, t0 + duration);
    const g = ac.createGain();
    g.gain.setValueAtTime(0.0001, t0);
    g.gain.exponentialRampToValueAtTime(0.1, t0 + 0.1);
    g.gain.exponentialRampToValueAtTime(0.0001, t0 + duration);
    src.connect(filter);
    filter.connect(g);
    g.connect(masterGain);
    src.start(t0);
}

```

Sonido de victoria (arpeggio)

Acorde ascendente para notificaciones de éxito, logros, ganadores.

```

function playWinSound() {
    const ac = getAudio();
    if (!ac) return;
    const notes = [523.25, 659.25, 783.99, 1046.5]; // C5 E5 G5 C6
    const t0 = ac.currentTime;
    notes.forEach((f, i) => {
        const o = ac.createOscillator();
        const g = ac.createGain();
        o.type = 'triangle';
        o.frequency.value = f;
        const start = t0 + i * 0.13;
        g.gain.setValueAtTime(0.0001, start);
        g.gain.exponentialRampToValueAtTime(0.18, start + 0.02);
        g.gain.exponentialRampToValueAtTime(0.0001, start + 0.5);
        o.connect(g);
        g.connect(masterGain);
        o.start(start);
        o.stop(start + 0.55);
    });
}

```

Copiar

Sonido de error

```
function playErrorSound() {
  const ac = getAudio();
  if (!ac) return;
  const o = ac.createOscillator();
  const g = ac.createGain();
  o.type = 'sawtooth';
  o.frequency.setValueAtTime(300, ac.currentTime);
  o.frequency.exponentialRampToValueAtTime(100, ac.currentTime + 0.3);
  g.gain.setValueAtTime(0.15, ac.currentTime);
  g.gain.exponentialRampToValueAtTime(0.0001, ac.currentTime + 0.3);
  o.connect(g);
  g.connect(masterGain);
  o.start();
  o.stop(ac.currentTime + 0.35);
}
```

Copiar

Sonido de notificación

```
function playNotif() {
  const ac = getAudio();
  if (!ac) return;
  const o = ac.createOscillator();
  const g = ac.createGain();
  o.type = 'sine';
  o.frequency.setValueAtTime(880, ac.currentTime);
  o.frequency.setValueAtTime(1100, ac.currentTime + 0.1);
  g.gain.setValueAtTime(0.15, ac.currentTime);
  g.gain.exponentialRampToValueAtTime(0.0001, ac.currentTime + 0.25);
  o.connect(g);
  g.connect(masterGain);
  o.start();
  o.stop(ac.currentTime + 0.3);
}
```

Copiar

Botón de silencio

```
let soundEnabled = true;
try {
  if (localStorage.getItem('my-sound') === '0') soundEnabled = false;
} catch (e) {}

function toggleSound() {
  soundEnabled = !soundEnabled;
  try { localStorage.setItem('my-sound', soundEnabled ? '1' : '0'); } catch (e) {}
}
```

Copiar

```
}

// Modifica getAudio para respetar el mute:
// Si (!soundEnabled) return null; al inicio de getAudio()
```

Canvas 2D

Dibujar formas, imágenes y animaciones en un canvas HTML5.

```
<canvas id="myCanvas" width="400" height="400"></canvas>
<script>
const canvas = document.getElementById('myCanvas');
const ctx = canvas.getContext('2d');

function draw() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  // Fondo
  ctx.fillStyle = '#1a1a2e';
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  // Círculo
  ctx.beginPath();
  ctx.arc(200, 200, 100, 0, Math.PI * 2);
  ctx.fillStyle = '#e94560';
  ctx.fill();
  ctx.strokeStyle = '#fff';
  ctx.lineWidth = 3;
  ctx.stroke();
}
draw();
</script>
```

[Copiar](#)

Animación con requestAnimationFrame

```
let rotation = 0;
let animating = false;

function animate() {
  if (!animating) return;
  rotation += 0.02;
```

[Copiar](#)

```
ctx.clearRect(0, 0, canvas.width, canvas.height);
ctx.save();
ctx.translate(200, 200);
ctx.rotate(rotation);
ctx.fillStyle = '#e94560';
ctx.fillRect(-50, -50, 100, 100);
ctx.restore();
requestAnimationFrame(animate);
}

function startAnimation() { animating = true; animate(); }
function stopAnimation() { animating = false; }
```

Dibujar imagen en círculo (clip)

```
function drawImageCircle(img, x, y, size) {
  ctx.save();
  ctx.beginPath();
  ctx.arc(x, y, size / 2, 0, Math.PI * 2);
  ctx.clip();
  ctx.drawImage(img, x - size / 2, y - size / 2, size, size);
  ctx.restore();
}
```

[Copiar](#)

Animación easeOut (frenado suave)

```
// Cubic ease-out: desaceleración suave
function easeOut(progress) {
  return 1 - Math.pow(1 - progress, 3);
}

// Uso en animación:
const duration = 2000; // ms
const start = Date.now();
function animate() {
  const elapsed = Date.now() - start;
  const progress = Math.min(elapsed / duration, 1);
  const eased = easeOut(progress);
  // eased va de 0 a 1 desacelerando al final
  if (progress < 1) requestAnimationFrame(animate);
}
animate();
```

[Copiar](#)



Grabación de video (Canvas a MP4)

Graba el contenido de un canvas y descarga el video como MP4 o WebM.

Grabación con MediaRecorder (recomendado)

```
let recRecorder = null;
let recChunks = [];
let recStream = null;

function startRecording(canvasElement, mime = 'video/mp4') {
  const stream = canvasElement.captureStream(30);
  recChunks = [];
  recRecorder = new MediaRecorder(stream, {
    mimeType: mime,
    videoBitsPerSecond: 12000000,
    audioBitsPerSecond: 128000 // si hay pista de audio
  });
  recRecorder.ondataavailable = (e) => {
    if (e.data && e.data.size) recChunks.push(e.data);
  };
  recRecorder.onstop = () => {
    const blob = new Blob(recChunks, { type: mime });
    recChunks = [];
    recRecorder = null;
    downloadBlob(blob, 'grabacion.' + (mime.includes('mp4') ? 'mp4' : 'webm'));
  };
  recRecorder.start(1000);
}

function stopRecording() {
  if (recRecorder && recRecorder.state !== 'inactive') recRecorder.stop();
}

function downloadBlob(blob, filename) {
  const a = document.createElement('a');
  a.href = URL.createObjectURL(blob);
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  setTimeout(() => { URL.revokeObjectURL(a.href); a.remove(); }, 1500);
}
```

[Copiar](#)

Grabación a mayor resolución (canvas offscreen)

REC_SCALE: Multiplicador de resolución (2 = 800x800 si el canvas visible es 400x400).

```
const REC_SCALE = 2; // 2x resolución
let recCanvas = null;

function ensureRecCanvas(displayCanvas) {
  if (!recCanvas) {
    recCanvas = document.createElement('canvas');
    recCanvas.width = displayCanvas.width * REC_SCALE;
    recCanvas.height = displayCanvas.height * REC_SCALE;
  }
  const c = recCanvas.getContext('2d');
  // Copiar el canvas de pantalla al de grabación
  c.drawImage(displayCanvas, 0, 0, recCanvas.width, recCanvas.height);
  return recCanvas;
}

// Al grabar, capturar del recCanvas:
// const stream = recCanvas.captureStream(30);
```

[Copiar](#)

Detectar soporte de formato

```
function detectRecorder() {
  if (typeof MediaRecorder !== 'undefined') {
    if (MediaRecorder.isTypeSupported('video/mp4;codecs=avc1.42E01E') ||
        MediaRecorder.isTypeSupported('video/mp4'))
      return 'mp4-media';
  }
  if (typeof MediaRecorder !== 'undefined') {
    if (MediaRecorder.isTypeSupported('video/webm;codecs=vp9') ||
        MediaRecorder.isTypeSupported('video/webm;codecs=vp8') ||
        MediaRecorder.isTypeSupported('video/webm'))
      return 'webm-media';
  }
  return null; // no soportado
}
```

[Copiar](#)

Grabación con audio del juego

Mezcla sonidos de WebAudio con el video del canvas en un solo archivo MP4.

```
// 1. Crear destino de audio para grabación
let recAudioDest = null;

function setupAudioForRecording() {
  const ac = getAudio(); // tu función getAudio
  if (!ac || !masterGain) return;
  if (!recAudioDest) {
    recAudioDest = ac.createMediaStreamDestination();
    masterGain.connect(recAudioDest); // captura todo lo que suena
  }
}

// 2. Combinar video + audio en un solo stream
function getCombinedStream(canvasElement) {
  setupAudioForRecording();
  const videoStream = canvasElement.captureStream(30);
  const audioTrack = recAudioDest
    ? recAudioDest.stream.getAudioTracks()[0]
    : null;
  return audioTrack
    ? new MediaStream([...videoStream.getVideoTracks(), audioTrack])
    : videoStream;
}

// 3. Usar el stream combinado con MediaRecorder
function startRecordingWithAudio(canvasElement) {
  const combined = getCombinedStream(canvasElement);
  recChunks = [];
  recRecorder = new MediaRecorder(combined, {
    mimeType: 'video/mp4;codecs=avc1.42E01E',
    videoBitsPerSecond: 12000000,
    audioBitsPerSecond: 128000
  });
  recRecorder.ondataavailable = (e) => {
    if (e.data && e.data.size) recChunks.push(e.data);
  };
  recRecorder.onstop = () => {
    const blob = new Blob(recChunks, { type: 'video/mp4' });
    recChunks = [];
    downloadBlob(blob, 'grabacion.mp4');
  };
  recRecorder.start(1000);
}

// 4. Limpiar al terminar
function cleanupRecording() {
  if (recRecorder && recRecorder.state !== 'inactive') recRecorder.stop();
}
```

```
if (recAudioDest && masterGain) {  
  try { masterGain.disconnect(recAudioDest); } catch (e) {}  
  recAudioDest = null;  
}  
}
```

Sistema de idiomas (i18n)

Traduce toda la interfaz con un objeto de traducciones y un selector de idioma.

```
// Objeto de traducciones  
const L = {  
  es: { greeting: 'Hola', goodbye: 'Adiós', action: 'Crear' },  
  en: { greeting: 'Hello', goodbye: 'Goodbye', action: 'Create' },  
  pt: { greeting: 'Olá', goodbye: 'Adeus', action: 'Criar' },  
  fr: { greeting: 'Bonjour', goodbye: 'Au revoir', action: 'Créer' },  
  de: { greeting: 'Hallo', goodbye: 'Auf Wiedersehen', action: 'Erstellen' }  
};  
  
let currentLang = 'es';  
  
function t(key) {  
  return L[currentLang]?.[key] || L['es']?.[key] || key;  
}  
  
// Detectar idioma del navegador  
function detectLang() {  
  const nav = (navigator.languages && navigator.languages[0])  
    ? navigator.languages[0]  
    : navigator.language;  
  const code = (nav || 'es').split('-')[0];  
  return L[code] ? code : 'es';  
}  
  
// Aplicar idioma a todos los elementos con data-i18n  
function setLanguage(lang) {  
  if (!L[lang]) lang = 'es';  
  currentLang = lang;  
  document.querySelectorAll('[data-i18n]').forEach(el => {  
    el.textContent = t(el.dataset.i18n);  
  });  
  document.querySelectorAll('[data-i18n-placeholder]').forEach(el => {  
    el.placeholder = t(el.dataset.i18nPlaceholder);  
  });  
}
```

[Copiar](#)

```

    });
    document.querySelectorAll('[data-i18n-title]').forEach(el => {
        el.title = t(el.dataset.i18nTitle);
    });
    try { localStorage.setItem('my-lang', lang); } catch (e) {}
    document.querySelectorAll('.lang-btn').forEach(btn => {
        btn.classList.toggle('active', btn.dataset.lang === lang);
    });
}

// Inicializar
const saved = localStorage.getItem('my-lang');
setLanguage(saved || detectLang());

```

HTML

```

<select id="langSelect" onchange="setLanguage(this.value)">
  <option value="es">ES Español</option>
  <option value="en">us English</option>
  <option value="pt">BR Português</option>
  <option value="fr">FR Français</option>
  <option value="de">DE Deutsch</option>
</select>

<h1 data-i18n="greeting">Hola</h1>
<p data-i18n="goodbye">Adiós</p>
<input data-i18n-placeholder="action" placeholder="Crear">

```

Copiar



Temas con CSS variables

Cambia toda la apariencia de la página con una línea de JavaScript.

CSS

```

/* Tema 1: Noche (oscuro) */
body[data-theme="noche"] {
  --bg: #0d1117;
  --card: #161b22;
  --text: #e6edf3;
  --accent: #58a6ff;
  --accent-soft: rgba(88,166,255,0.15);
}

```

Copiar

```
--border: #30363d;
}

/* Tema 2: Claro */
body[data-theme="claro"] {
  --bg: #ffffff;
  --card: #f6f8fa;
  --text: #24292f;
  --accent: #0969da;
  --accent-soft: rgba(9,105,218,0.1);
  --border: #d0d7de;
}

/* Tema 3: Esmeralda */
body[data-theme="esmeralda"] {
  --bg: #0a1f1a;
  --card: #0f2d25;
  --text: #c8e6d8;
  --accent: #3fb950;
  --accent-soft: rgba(63,185,80,0.15);
  --border: #1a3a2a;
}

/* Usar las variables */
body {
  background: var(--bg);
  color: var(--text);
}
.card {
  background: var(--card);
  border: 1px solid var(--border);
}
a { color: var(--accent); }
```

JavaScript

```
function setTheme(theme) {
  document.body.dataset.theme = theme;
  document.querySelectorAll('.theme-btn').forEach(btn => {
    btn.classList.toggle('active', btn.dataset.theme === theme);
  });
  try { localStorage.setItem('my-theme', theme); } catch (e) {}
}

// Cargar tema guardado
try {
```

[Copiar](#)

```
const saved = localStorage.getItem('my-theme');  
if (saved) setTheme(saved);  
} catch (e) {}
```

Drag & Drop de archivos

Permite arrastrar archivos o seleccionarlos con clic, validando tipo y cantidad.

```
<div id="dropZone" class="drop-zone">  
  Arrastra archivos aquí  
</div>  
<input type="file" id="fileInput" multiple accept="image/*" style="display:none">  
  
<script>  
const dropZone = document.getElementById('dropZone');  
const fileInput = document.getElementById('fileInput');  
const MAX_FILES = 10;  
  
function handleFiles(files) {  
  const valid = [];  
  for (const f of files) {  
    if (!f.type.startsWith('image/')) continue;  
    if (valid.length >= MAX_FILES) break;  
    valid.push(f);  
  }  
  if (valid.length === 0) return;  
  console.log('Archivos válidos:', valid);  
  // Procesar archivos...  
}  
  
dropZone.addEventListener('click', () => fileInput.click());  
  
dropZone.addEventListener('dragover', (e) => {  
  e.preventDefault();  
  dropZone.classList.add('dragover');  
});  
  
dropZone.addEventListener('dragleave', () => {  
  dropZone.classList.remove('dragover');  
});  
  
dropZone.addEventListener('drop', (e) => {  
  e.preventDefault();
```

[Copiar](#)

```
dropZone.classList.remove('dragover');
handleFiles(e.dataTransfer.files);
});

fileInput.addEventListener('change', (e) => {
  handleFiles(e.target.files);
});
</script>

<style>
.drop-zone {
  border: 2px dashed var(--border);
  border-radius: 12px;
  padding: 40px;
  text-align: center;
  cursor: pointer;
  transition: all 0.3s;
}
.drop-zone:hover,
.drop-zone.dragover {
  border-color: var(--accent);
  background: var(--accent-soft);
}
</style>
```



Procesamiento de imágenes

Redimensiona imágenes subidas para optimizar rendimiento.

```
const MAX_DIMENSION = 800;

function processImage(file, callback) {
  const reader = new FileReader();
  reader.onerror = () => callback(null);
  reader.onload = (e) => {
    const img = new Image();
    img.onerror = () => callback(null);
    img.onload = () => {
      const w = img.width;
      const h = img.height;
      if (w <= MAX_DIMENSION && h <= MAX_DIMENSION) {
        callback(img);
        return;
      }
    }
  }
}
```

[Copiar](#)

```
    }
    const scale = MAX_DIMENSION / Math.max(w, h);
    const cw = Math.round(w * scale);
    const ch = Math.round(h * scale);
    const c = document.createElement('canvas');
    c.width = cw;
    c.height = ch;
    c.getContext('2d').drawImage(img, 0, 0, cw, ch);
    const resized = new Image();
    resized.onload = () => { callback(resized); URL.revokeObjectURL(resized);
    resized.src = c.toDataURL('image/webp', 0.85);
  };
  img.src = e.target.result;
};
reader.readAsDataURL(file);
}

// Crear miniatura
function makeThumb(img, size = 128) {
  const scale = size / Math.max(img.width, img.height);
  const w = Math.max(1, Math.round(img.width * scale));
  const h = Math.max(1, Math.round(img.height * scale));
  const c = document.createElement('canvas');
  c.width = w;
  c.height = h;
  c.getContext('2d').drawImage(img, 0, 0, w, h);
  return c;
}
```



Persistencia en localStorage

Guarda y carga configuración del usuario de forma local (sin servidor).

```
// Guardar
function save(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
    return true;
  } catch (e) {
    return false;
  }
}
```

[Copiar](#)

```
// Cargar
function load(key, fallback = null) {
  try {
    const raw = localStorage.getItem(key);
    return raw !== null ? JSON.parse(raw) : fallback;
  } catch (e) {
    return fallback;
  }
}

// Eliminar
function remove(key) {
  try { localStorage.removeItem(key); } catch (e) {}
}

// Ejemplo de uso:
save('my-settings', { theme: 'noche', lang: 'es', volume: 80 });
const settings = load('my-settings', { theme: 'claro', lang: 'en' });
```



Aleatoriedad segura (crypto)

Genera números aleatorios criptográficamente seguros para sorteos, ruletas, etc.

```
// Número aleatorio entre 0 (incl.) y 1 (excl.) – seguro
function cryptoRand() {
  const buf = new Uint32Array(1);
  crypto.getRandomValues(buf);
  return buf[0] / 4294967296;
}

// Entero aleatorio entre min (incl.) y max (incl.)
function cryptoInt(min, max) {
  return min + Math.floor(cryptoRand() * (max - min + 1));
}

// Elegir elemento aleatorio de un array
function cryptoChoice(arr) {
  return arr[Math.floor(cryptoRand() * arr.length)];
}

// Mezclar array (Fisher-Yates seguro)
function cryptoShuffle(arr) {
```

[Copiar](#)

```
const a = [...arr];
for (let i = a.length - 1; i > 0; i--) {
  const j = Math.floor(cryptoRand() * (i + 1));
  [a[i], a[j]] = [a[j], a[i]];
}
return a;
}

// Ejemplo: elegir ganador de un array
const participantes = ['Ana', 'Bob', 'Carlos', 'Diana'];
const ganador = cryptoChoice(participantes);
```

Firebase Auth (Google Login)

Iniciar sesión con Google para identificar al usuario.

HTML

```
<div id="authPanel">
  <div id="userChip" style="display:none">
    <img id="userAvatar" src="" alt="" width="30" height="30"
      style="border-radius:50%">
    <span id="userName"></span>
    <button onclick="doLogout()">Cerrar sesión</button>
  </div>
  <button id="googleBtn" onclick="doLogin()">
    Iniciar sesión con Google
  </button>
</div>
```

[Copiar](#)

JavaScript

```
// Firebase compat (insertar en HTML)
// <script src="https://www.gstatic.com/firebasejs/10.12.2/firebase-app-compat.js">
// <script src="https://www.gstatic.com/firebasejs/10.12.2/firebase-auth-compat.js">

const firebaseConfig = {
  apiKey: "TU_API_KEY",
  authDomain: "TU_PROYECTO.firebaseio.com",
  projectId: "TU_PROYECTO",
  storageBucket: "TU_PROYECTO.appspot.com",
```

[Copiar](#)

```
messagingSenderId: "123456789",
appId: "1:123456789:web:abcdef"
};
firebase.initializeApp(firebaseConfig);
const fbAuth = firebase.auth();
const provider = new firebase.auth.GoogleAuthProvider();

function doLogin() {
  fbAuth.signInWithPopup(provider).catch((e) => {
    console.error('Error login:', e);
  });
}

function doLogout() {
  fbAuth.signOut();
}

fbAuth.onAuthStateChanged((user) => {
  if (user) {
    document.getElementById('googleBtn').style.display = 'none';
    document.getElementById('userChip').style.display = 'flex';
    document.getElementById('userName').textContent = user.displayName;
    document.getElementById('userAvatar').src = user.photoURL || '';
  } else {
    document.getElementById('googleBtn').style.display = 'block';
    document.getElementById('userChip').style.display = 'none';
  }
});
```



Firestore (base de datos)

Guardar y cargar datos por usuario (configuración, sets, historial).

```
// <script src="https://www.gstatic.com/firebasejs/10.12.2/firebase-firestore-compat

const fbDB = firebase.firestore();

// Guardar documento
async function saveDoc(collection, docId, data) {
  try {
    await fbDB.collection(collection).doc(docId).set(data, { merge: true });
    return true;
  }
}
```

Copiar

```
    } catch (e) {
      console.error('Error guardando:', e);
      return false;
    }
  }

// Cargar documento
async function loadDoc(collection, docId) {
  try {
    const snap = await fbDB.collection(collection).doc(docId).get();
    return snap.exists ? snap.data() : null;
  } catch (e) {
    console.error('Error cargando:', e);
    return null;
  }
}

// Cargar todos los documentos de un usuario
async function loadUserDocs(collection, uid) {
  try {
    const snap = await fbDB.collection(collection)
      .where('uid', '==', uid)
      .orderBy('createdAt', 'desc')
      .get();
    return snap.docs.map(d => ({ id: d.id, ...d.data() }));
  } catch (e) {
    console.error('Error listando:', e);
    return [];
  }
}

// Eliminar documento
async function deleteDoc(collection, docId) {
  try {
    await fbDB.collection(collection).doc(docId).delete();
    return true;
  } catch (e) {
    console.error('Error eliminando:', e);
    return false;
  }
}

// Ejemplo: guardar set de imágenes
async function saveImageSet(uid, name, images) {
  const docId = uid + '_' + Date.now();
  await saveDoc('imageSets', docId, {
    uid, name, images,
    createdAt: firebase.firestore.FieldValue.serverTimestamp()
  });
}
```

```
});  
}
```

⚡ Resumen rápido — Funciones por categoría

[Copiar](#)

CATEGORÍA	FUNCIONES
 Audio	<code>getAudio()</code> , <code>playTick()</code> , <code>playWhoosh()</code> , <code>playWinSound()</code> , <code>playErrorSound()</code> , <code>playNotif()</code> , <code>toggleSound()</code>
 Canvas	<code>drawImageCircle()</code> , <code>easeOut()</code> , <code>requestAnimationFrame</code> loop
 Grabación	<code>startRecording()</code> , <code>stopRecording()</code> , <code>ensureRecCanvas()</code> , <code>detectRecorder()</code> , <code>getCombinedStream()</code> (con audio)
 Idiomas	<code>t()</code> , <code>setLanguage()</code> , <code>detectLang()</code>
 Temas	<code>setTheme()</code> , CSS variables
 Drag & Drop	<code>handleFiles()</code> , drop zone listeners
 Imágenes	<code>processImage()</code> , <code>makeThumb()</code>
 localStorage	<code>save()</code> , <code>load()</code> , <code>remove()</code>
 Criptografía	<code>cryptoRand()</code> , <code>cryptoInt()</code> , <code>cryptoChoice()</code> , <code>cryptoShuffle()</code>
 Firebase Auth	<code>doLogin()</code> , <code>doLogout()</code>
 Firestore	<code>saveDoc()</code> , <code>loadDoc()</code> , <code>loadUserDocs()</code> , <code>deleteDoc()</code>

